

Plan de migración de base de datos y datos

CELUACCEL

ANÁLISIS Y DESARROLLO DE SOFTWARE

**JHONATAN COMEZAQUIRA
NICOLAS MONTENEGRO
ANDERSON JOYA**

1. Objetivo

Migrar la base de datos del Sistema Celuacel de MySQL a PostgreSQL para garantizar una integridad referencial estricta, optimizar el control de concurrencia multiversión (MVCC) y cumplir con los rigurosos estándares de aseguramiento de calidad del programa ADSO.

2. Alcance

Alcance Abarca la base de datos CELUACCEL en los ambientes de desarrollo y pruebas. Involucra la totalidad de las tablas principales: usuarios, roles, inventario (repuestos), órdenes de servicio e historial de chat.

3. Situación actual

Motor origen: MySQL (versión 8.x).

Tamaño y registros: Entorno de pruebas con un volumen ligero Relaciones:
Implementación de llaves foráneas básicas (ej. vinculación de la tabla usuarios con roles, y órdenes de servicio con técnicos).

4. Situación destino

Motor destino: PostgreSQL versión 16 o superior.

Ambiente y estructura: Servidor local o contenedor, manteniendo la arquitectura relacional exacta, pero aprovechando tipos de datos nativos más avanzados.

5. Inventario de datos

Se migrarán los perfiles de los actores (clientes, técnicos, administradores), credenciales encriptadas, catálogo de repuestos con sus cantidades de stock vigentes, tickets de reparación con sus diagnósticos, y el historial de mensajes de soporte.

6. Estrategia de migración

Exportación de los datos estructurales mediante un respaldo (MySQL DUMP). Transformación manual o asistida por herramientas ETL de los scripts SQL para adaptar la sintaxis, seguido de una importación limpia (Restore) en PostgreSQL a través de la consola.

7. Mapeo de datos

- tb_usuarios.id (INT AUTO_INCREMENT) -> usuarios.id (SERIAL o IDENTITY)
- tb_usuarios.id_rol (TINYINT) -> usuarios.rol_id (INTEGER)
- tb_ordenes.estado (VARCHAR) -> ordenes_servicio.estado (Tipo ENUM nativo)

8. Transformaciones

- Booleanos: Conversión de campos de estado lógico (como usuario_activo) de TINYINT(1) en MySQL a tipo BOOLEAN nativo en PostgreSQL.
- Fechas: Ajuste de los campos DATETIME a TIMESTAMP WITH TIME ZONE para manejar la trazabilidad temporal con mayor precisión.

9. Orden de migración

Para no violar las restricciones de llaves foráneas, la inserción se ejecutará así:

1. Tablas maestras (Roles, Catálogo de estados).
2. Usuarios.
3. Inventario (Repuestos y accesorios).
4. Transaccionales (Órdenes de servicio).
5. Dependientes (Historial de chat).

10. Pruebas

- Antes: Validación de los scripts DDL de creación en un esquema PostgreSQL vacío.
- Después: Ejecución de pruebas unitarias (con Jest) y pruebas E2E (con Playwright) contra la API en Express para certificar que el inicio de sesión y la actualización de stock funcionen correctamente.

11. Validación

Comparación de la cantidad de registros (SELECT COUNT(*)) entre el origen y el destino. Verificación de la integridad de los datos críticos e inspección con herramientas de análisis estático, como SonarCloud, para garantizar que no existan sentencias SQL incompatibles en el código del backend.

12. Ventana de migración

Se programará durante un periodo de inactividad operativa. (Ejemplo: Sábado a las 10:00 PM). La duración estimada es de 2 horas.

13. Plan de contingencia / rollback

La base de datos original en MySQL se mantendrá intacta y operativa. Si la migración presenta fallos críticos, se cambiará inmediatamente la cadena de conexión en el archivo .env del backend (Node.js) apuntando de vuelta al puerto de MySQL para restablecer el servicio.

14. Responsables

Ejecución técnica y transformación: Anderson Joya.

-Verificación y validación de testing: Nicolas Montenegro y Jhonatan Comezaquira.

-Autorización: Instructor líder del proyecto.

15. Evidencias

Logs de consola del proceso de exportación/importación, capturas de pantalla de la estructura importada, y los reportes en verde de la cobertura de pruebas de los endpoints garantizando la correcta comunicación con la nueva base de datos.

CRITERIOS DE ACEPTACIÓN DE BASE DE DATOS

Criterio de Aceptación (QA)	MySQL (Motor Actual)	PostgreSQL (Motor a Migrar)
Integridad y Restricciones	Presenta mayor flexibilidad en el manejo de restricciones o sintaxis.	Garantiza una extrema robustez e integridad estricta de los datos.
Control de Concurrencia	Maneja las consultas simples eficientemente mediante bloqueos a nivel de fila.	Emplea Control de Concurrencia Multiversión (MVCC), permitiendo que múltiples usuarios lean y escriban simultáneamente sin generar bloqueos.
Soporte de Estructuras	Su soporte para formatos como JSON es funcional.	Ofrece soporte nativo y avanzado para estructuras complejas como JSONB o Arrays.
Lógica y Rendimiento	Sistema puramente relacional enfocado en ofrecer alta velocidad en operaciones de lectura.	Arquitectura objeto-relacional superior para soportar y ejecutar lógicas de negocio complejas.

